

AI work

I build AI agents that do engineering work, and the checks that decide whether that work is accepted.

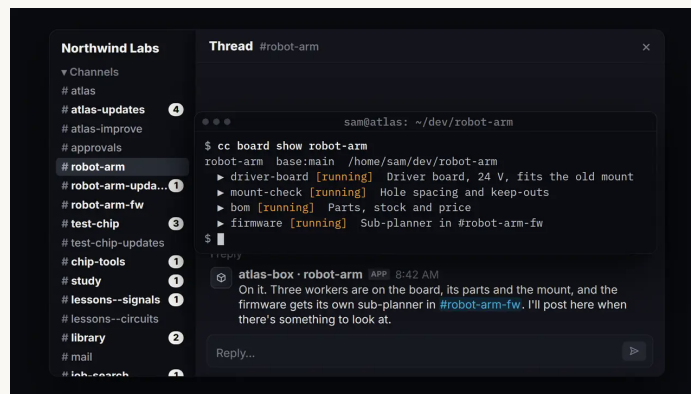
I'm a second-year electrical engineering student at Waterloo. Since March I've been building with Claude Code: agents that write and land software, design circuit boards and lay out chip blocks, and tools that turn my courses into lessons and notes. The pattern is the same in each. The agent does the work, deterministic tools check it, a failed check sends it back with the reason, and anything irreversible, like paying for a board, waits for a person.

Each section below covers one project: what it does, how it works, and how far it has actually got.

autobox

github.com/ihsan-sa/autobox

autobox runs Claude Code on my projects while nobody is watching, and I steer it from my phone over Slack. It runs my projects, my coursework and my job search from a small server at home. I wrote about how I built it and how I use it day to day in a separate essay, so here I'll stick to what makes it safe to leave alone.



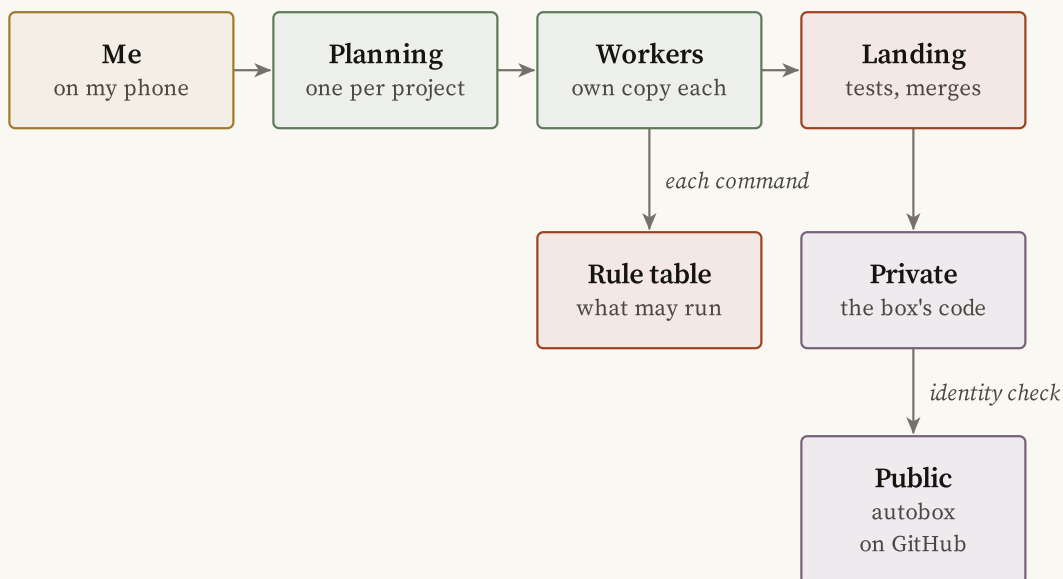
autobox at work from the phone, a Slack thread through to a merged PR

Claude Code is good at one sitting of work. Left running for days it has three problems: its context fills up and it forgets, it can run any command it likes, and nothing stops it merging broken code. autobox is the layer around it that fixes those three. It isn't another agent. It's shell and Python glue around Claude Code.

A planning session for each project splits the work into tasks. Each worker runs in its own git worktree as a loop of fresh-context iterations, keeping its state in a brief and a journal on disk. Every iteration is a new process that reads the brief and the end of the journal, does the next unfinished step, and writes a dated entry early, so an iteration cut short still leaves the next one a trail. The worker never commits. A hook commits its work, and a blocklist checks every shell command before it runs.

The first version of that blocklist matched rules against the command as typed, so `setuid tmux kill-server` slipped past a rule written for `tmux kill-server`. Now every rule is checked against the command plus every suffix of every statement in it, so a wrapper word in front no longer hides anything. A change to the rules is tested by replaying the commands agents really ran through the old and new versions and reading what newly gets refused.

One program merges, and only after the checks pass on the branch merged into current main. A branch can pass its own tests and still break main once it meets work that landed after it branched, and here nobody reads most of the diffs before they merge. After a merge the checks run again on the new tip, and only a green tip is installed. Because the lander is the one program that can merge, it refuses any pull request that touches its own files. Those go through a separate tool that shares none of its code and needs a security review and a second review on record.



In its first five weeks it merged about 790 pull requests into its own code. The blocklist stops accidents and casual prompt injection, not a determined attacker, and it doesn't catch every command that has the same effect under a different name. Those are left to review. The gate suite also still runs on the machine itself, because hosted CI minutes for a private repo aren't free.

How the agents write to me

github.com/ihsan-sa/autobox

Everything the agents do reaches me as text on my phone, so how they write is part of the system. Their posts used to read like a model told to be brief: stacked nouns, task names, the brief pasted back into every pull request. My theory was that writing can only be compressed so far before it costs the reader more to decode than it saves. So I had the agents study my writing and the technical-writing textbooks, and turned what they found into rules every session carries.

I ran the change as an experiment and tested the new rules blind on twelve real cases. In the first round a model judge preferred the old, shorter versions 10 times out of 12. Shown the pairs unlabelled, I picked the new ones in all four I judged, and I went with mine. A week later, posts that said "I" had gone from 18% to 53%, the median pull-request description had halved, and almost none pasted the brief back any more.

A second document sets what reaches me at all: what changed, what is ready and what only I can decide, with the rest left in logs and journals. An audit then found I had answered none of the 83 task threads opened in my main channel, so they moved to a side channel. In a replay of three days, that cut the main channel's posts by about a third.

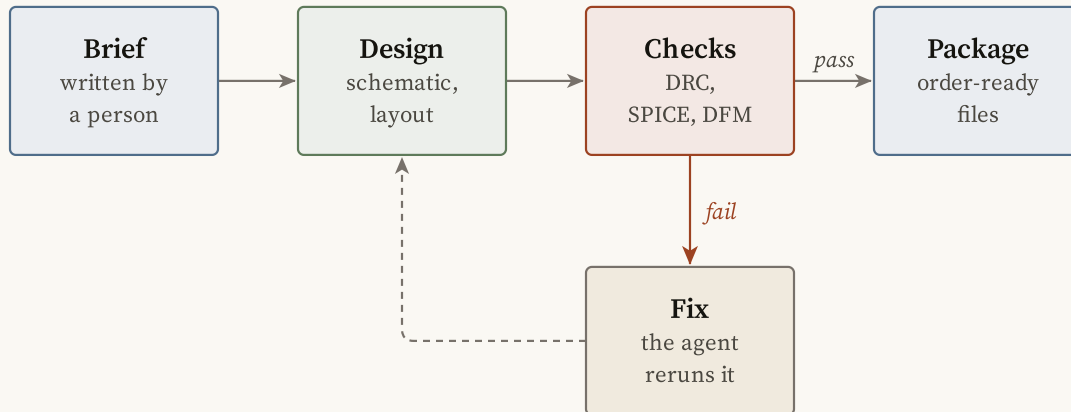
The rules are public, in autobox's `docs/WRITING.md` and `docs/comms-contract.md`.

hwde

github.com/ihsan-sa/hwde

hwde turns a plain-English brief for a circuit board into the files a factory needs. It's one Claude Code skill that dispatches to about two dozen subagents and a larger set of Python scripts. The subagents make the design decisions: they read a datasheet, choose a topology, place a part, or decide whether a violation is real. The scripts run every check that can be computed, from

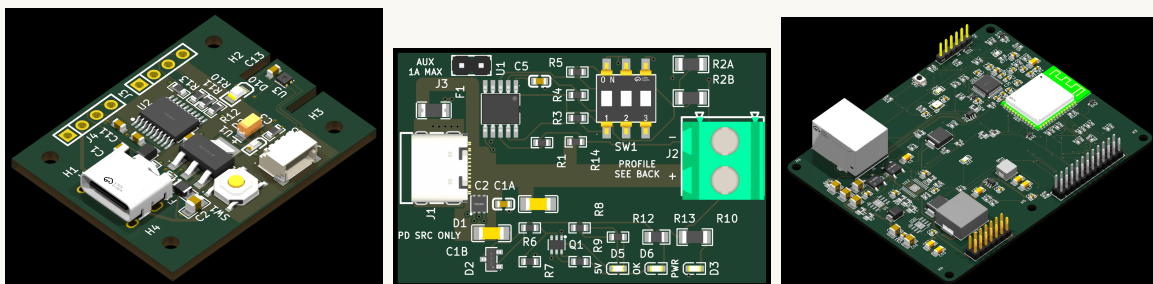
KiCad's design-rule check to ngspice simulation. The split matters because a subagent can be wrong in a way that sounds confident, and a script either passes or it doesn't.



A board goes through intake and research, parts and schematic, placement and routing, verification, and ordering, with a checkpoint for a person at each. A failed check becomes work orders for fixer agents. When a design-rule check lists dozens of violations, a script clusters them by net and region, so one fixer gets one coherent job instead of twenty fixers each getting a line. A board that fails the same gate twice goes to a person instead of looping again.

The checks are themselves tested against boards with planted faults. Three clean boards carry twelve faults planted one at a time, such as a diode turned round, an undersized power trace or a plane slotted under an RF feed, and each fault names the check that must catch it. Each of the ten electrical checks catches the fault planted for it. The current-capacity check still raises mostly false alarms.

One unattended run took a brief to files ready to order in about fourteen hours, and stopped there because paying needs a person. The ordering script is the only one that spends money, and it wants a fresh quote, a design hash that matches what's on disk, and a typed confirmation of the board, the quantity and the total.



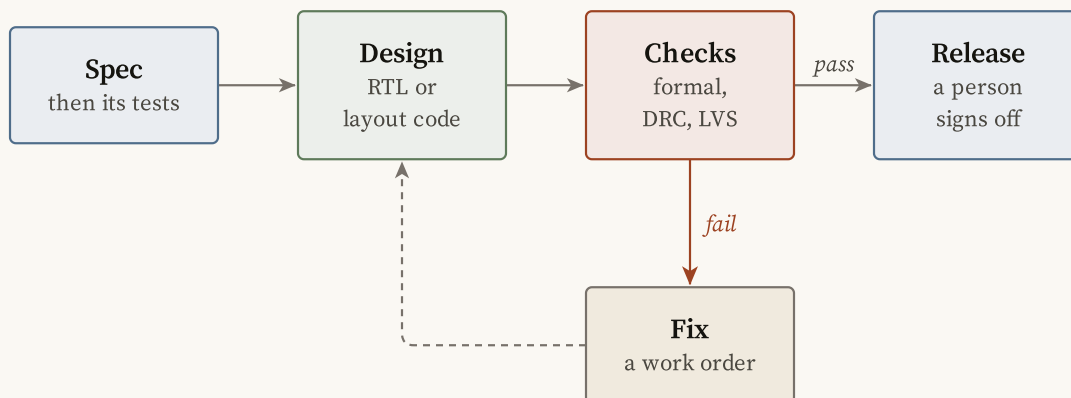
KiCad renders of three boards hwde designed

Two boards it designed have been made, though neither has been powered on yet, so nothing here says a board works. One of them was ordered by hand before its verify gate passed. Reading its code, I found that an agent could record a person's sign-off for itself. I haven't seen one do it in a run, but the code as written would allow it with a single command, and the fix is to move that call somewhere an agent's own commands can't reach.

chip-flow

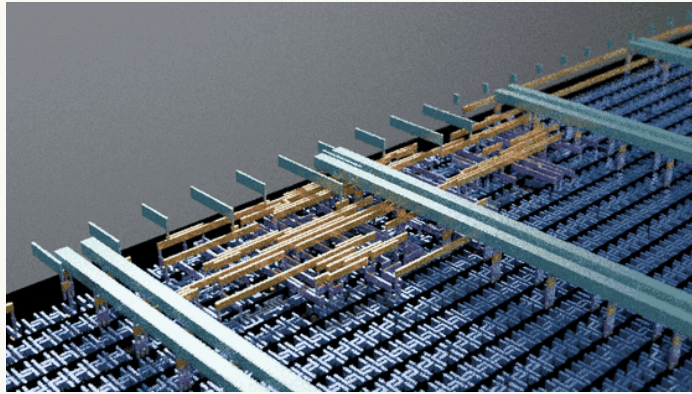
github.com/ihsan-sa/chip-flow

chip-flow takes the same approach to chip blocks, on open tools and the GF180MCU process. It's hwde's engine ported to chip design, aimed at Verilog and analog netlists instead of KiCad boards. One engine sits under three skills: digital, analog and mixed-signal. The mixed-signal skill splits a spec into a digital and an analog side, runs each through the other two skills, then joins them and checks the join in one co-simulation.



A digital block's testbench is written before its RTL, by an agent that has read only the spec, and the testbench is itself tested with mutants. A testbench that asserts nothing will pass any design, so the visible tests must kill at least 90% of a fixed set of mutants of the RTL. The design must also pass tests it never saw. For the scored runs, those tests are kept outside any workspace and run only after the agent's run has ended.

There's no analog autorouter, so an analog layout is a Python script that draws it. Magic, KLayout and netgen only judge the result, with DRC, LVS and a simulation of the extracted layout. A DRC finding becomes a work order for the script, so the fix goes into the code and the layout can be rebuilt from it.



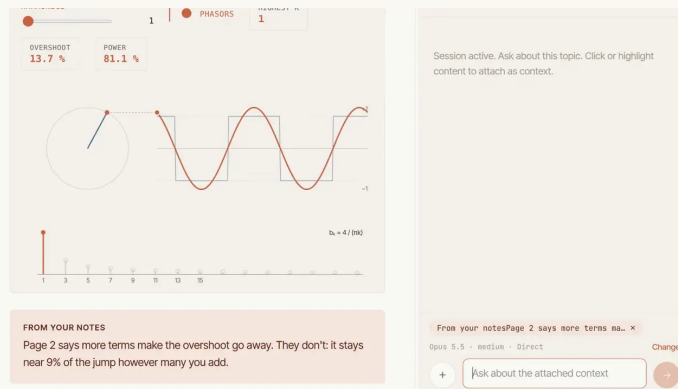
the 8-bit counter's GF180 layout in 3D

An 8-bit counter and a UART have gone from spec to a hardened, timed netlist with every check green, and the UART's testbench kills every mutant. A current mirror and a comparator have gone to a checked layout, though those two runs were told to take the default at both checkpoints, so no person signed them. A mixed-signal example went green on every check, but I don't count it, because two of its mutant rulings were edited by hand. The SPI FIFO is stuck at its formal proof, which times out before it can close. Nothing has been sent to a fab yet.

lesson-builder

github.com/ihsan-sa/lesson-builder

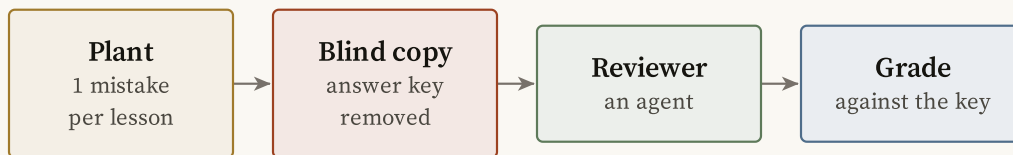
lesson-builder builds the interactive web lessons I study from, each with a tutor inside it. It was the first agent system I built, to study for my exams. A lesson is a small web app with LaTeX for notation, SVG for graphs, manim for animation, and a chat panel where the tutor answers questions about it.



a lesson on Fourier series with its tutor drawing into the chat

Specialist agents research, plan, draw and build demos. I approve the plan once, and the pipeline can't skip that step. Separate reviewers then check the code, the teaching, the visuals and the science, each on its own so no review softens because another passed. Each verdict is recorded against a hash of the files it read, so an update skips anything that passed and hasn't changed. That's what makes it affordable to re-check every diagram and demo on every update.

The tutor can propose edits to the lesson while a student reads it, and an edit lands only after code review passes. It runs confined to its lesson, because early tutors acted on instructions meant for the machine they ran on. In one course a tutor wrote a handoff entry to a file outside its lesson and told the student "Handoff entry written." Every tutor now runs with no project instructions, hooks or MCP servers loaded, and its shell runs in a sandbox that can write only a scratch directory. If the installed Claude Code can't enforce all of that, the tutor doesn't start.



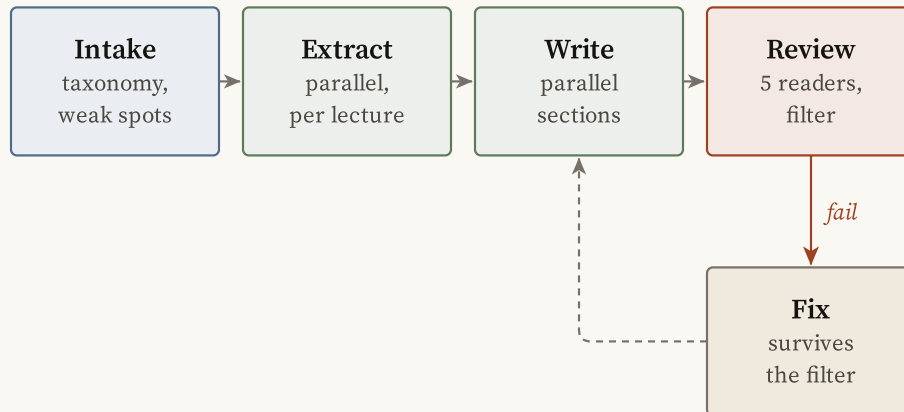
I tested the teaching reviewer blind on 24 planted mistakes with the answer key withheld, and in August it caught all of them. The rules it enforces have changed a little since, so it's due to run again. There's no multi-turn tutor eval yet, and the tutor's edits still travel as tags in model text, parsed by regular expressions, where a typed action channel would be sturdier.

pdf-material-builder

github.com/ihsan-sa/pdf-material-builder

pdf-material-builder turns a course into LaTeX study documents in one look and one voice: reference docs, formula sheets, worked examples, course notes and technical write-ups. It built the PDF overview and the five technical notes this page draws on.

Left to separate write-ups, these documents drift. One uses a different symbol for the same quantity than the next, and the voice shifts with whoever wrote the section. For a large course, agents read the lectures in parallel, one conventions file fixes the notation, and every writer reads that file first. The reference doc is written before the rest, so the documents after it have something settled to point back to.



Five reviewers then read the result at once. One plays a past student who got a B or C, one checks every document against a table of canonical equations, and one re-derives the math in SymPy and reports pass, fail or unverifiable for each claim. A filter throws out the findings that don't survive a re-check: it re-derives a math claim before believing it, and it needs two reviewers to agree before a complaint about style counts. Only what survives goes back to a writer.

It still can't take a Word document to a PDF, because the pipeline has no LibreOffice to convert it.

I'm looking for a January to April 2027 co-op, and you can reach me through ihsan.cc.